UNIVERSIDAD DE COSTA RICA
SEDE DEL ATLÁNTICO - RECINTO PARAÍSO
CARRERA DE INFORMÁTICA EMPRESARIAL
CURSO: IF0009 - Desarrollo de Software IV
PROFESOR: Mag. Jonathan Granados C.
SEMESTRE: II-2026

Práctica Guiada 13: Construcción de la Primera Aplicación Móvil con Flutter SDK y Resolución Integral de Errores de Virtualización y Configuración

Resumen

Esta práctica guiada constituye la primera inmersión práctica en el desarrollo móvil multiplataforma del curso. A partir de las directrices de ingeniería establecidas en la `GUIA_PASO_A_PASO_FLUTTER_SETUP_Y_DESARROLLO.md`, usted aprenderá a verificar su entorno de desarrollo, diagnosticar fallas tempranas mediante `flutter doctor`, superar los bloqueos de configuración más comunes del sistema operativo anfitrión (incluyendo incompatibilidades de virtualización en Windows con Hyper-V, WSL2 y emuladores AVD) y construir paso a paso su primera aplicación móvil funcional: **FocusTrack Móvil** (Gestor de Productividad y Metas Diarias).

La práctica combina explicaciones minuciosas, código fuente 100% tipado y libre de advertencias en Dart 3, lineamientos de diseño moderno con **Material 3**, protección de interfaz en pantallas con muesca (*notch*) y barras de gestos mediante `SafeArea`, y el flujo de productividad acelerada con **Stateful Hot Reload**.

Metadatos del Laboratorio

- **Tiempo estimado de desarrollo:** 3 horas y 30 minutos (210 minutos).
- **Herramientas requeridas:** Flutter SDK 3.22+, Dart SDK 3.4+, Visual Studio Code con extensiones oficiales Dart & Flutter, Android Studio (gestor de SDK y AVD), Java OpenJDK 17 LTS o 21 LTS, Git 2.40+.

  Metas de Aprendizaje:

  Comprobar y diagnosticar el estado del toolchain móvil multiplataforma, resolviendo metódicamente errores de licencias, rutas del sistema y bloqueos de virtualización por hardware (Hyper-V, WHPX, BIOS SVM/VT-x).

  Inicializar un proyecto Flutter estructurado, reconociendo la función arquitectónica de cada carpeta nativa (`android/`, `ios/`) y el rol central de Dart en `lib/main.dart`.

  Implementar una aplicación móvil interactiva utilizando Material 3, gestión de estado local reactivo (`StatefulWidget`, `setState()`), componentes responsivos (`Scaffold`, `Card`, `FloatingActionButton`) y experimentar el ciclo de Stateful Hot Reload en tiempo real.

Ficha de Requisitos Previos y Verificación de Entorno

Antes de comenzar la codificación, verifique que los siguientes componentes estén instalados en su estación de trabajo:

```
# 1. Comprobar version de Git
git --version
```

```
# 2. Comprobar version de Java (Requerido JDK 17 o 21)
java -version

# 3. Comprobar version de Flutter SDK
flutter --version
```

Salida esperada aproximada: * `git version 2.40.0` o superior. * `openjdk version "17.0.x"` o `"21.0.x"`. * `Flutter 3.24.x • channel stable` / `Dart 3.5.x`.

Parte 1: Práctica Guiada - Construcción Paso a Paso de "FocusTrack Móvil"

En esta sección construirá desde cero la aplicación móvil **FocusTrack Móvil**, una herramienta de productividad que permite registrar metas completadas, reiniciar contadores de sesión, recibir alertas visuales y alternar estados de prioridad con animaciones de interfaz.

Paso 1.1: Diagnóstico de Estado con `flutter doctor`

Abra una consola de PowerShell o la terminal integrada de Visual Studio Code y ejecute el comando de telemetría y salud del SDK:

```
flutter doctor -v
```

Analice la salida en pantalla. Para continuar sin contratiempos con el desarrollo de Android, debe visualizar marcas de verificación verdes (`[✓]`) en los siguientes tres rubros obligatorios:

```
[✓] Flutter (Channel stable, 3.24.3, on Microsoft Windows ...)
[✓] Android toolchain - develop for Android devices (Android SDK version 34.0.0)
[✓] VS Code (version 1.94.x)
[✓] Connected device (1 available)
```

> [!NOTE] Si observa advertencias (`[!]`) o errores (`[✗]`) relacionados con licencias de Android, componentes de línea de comandos o emuladores, continúe leyendo; en la **Parte 2** de esta guía aprenderá el procedimiento exacto de resolución.

Paso 1.2: Creación Automatizada del Proyecto Móvil

Cree el espacio de trabajo de la aplicación especificando de forma determinista las plataformas móviles soportadas:

```
# Navegue a su carpeta habitual de proyectos universitarios
cd C:\Users\jonat\OneDrive\Documentos\UCR-2026\IF0009

# Cree el proyecto con plataformas moviles exclusivas
flutter create --platforms=android,ios focus_track_movil

# Ingrese al directorio de la solucion
cd focus_track_movil

# Abra el proyecto en Visual Studio Code
code .
```

Paso 1.3: Inspección de la Anatomía del Proyecto

Explore el explorador de archivos en el panel lateral izquierdo de VS Code. La estructura del proyecto Flutter se organiza de la siguiente manera:

```
focus_track_movil/
 ■■■ android/              <-- Contenedor nativo Android (Gradle, Manifest)
 ■■■ ios/                  <-- Contenedor nativo iOS (Xcode, Info.plist)
 ■■■ lib/                  <-- CODIGO FUENTE COMPARTIDO EN DART
 ■    ■■■ main.dart        <-- Punto de entrada principal de la aplicacion
 ■■■ test/                 <-- Pruebas de widgets y logica unitaria
 ■■■ pubspec.yaml          <-- Declaracion de dependencias y fuentes
 ■■■ analysis_options.yaml<-- Reglas de linter estatico de Dart
```

Paso 1.4: Implementación del Código Fuente en `lib/main.dart`

Abra el archivo `lib/main.dart`, elimine el código de demostración generado por defecto y reemplace su contenido en su totalidad por el siguiente código robusto, tipado, autodocumentado y basado en el estándar moderno **Material 3**:

```
import 'package:flutter/material.dart';

void main() {
 runApp(const FocusTrackApp());
}

/// Widget raiz inmutable que configura el tema visual de la aplicacion
class FocusTrackApp extends StatelessWidget {
 const FocusTrackApp({super.key});

 @override
 Widget build(BuildContext context) {
   return MaterialApp(
     title: 'FocusTrack Móvil',
     debugShowCheckedModeBanner: false,
     theme: ThemeData(
       useMaterial3: true,
       colorScheme: ColorScheme.fromSeed(
         seedColor: const Color(0xFF1A237E), // Azul Índigo Universitario
         brightness: Brightness.light,
       ),
       appBarTheme: const AppBarTheme(
         centerTitle: true,
         elevation: 2,
       ),
     ),
     home: const PantallaPrincipalMetas(),
   );
 }
}

/// Pantalla con estado reactivo para registrar metas y productividad
class PantallaPrincipalMetas extends StatefulWidget {
 const PantallaPrincipalMetas({super.key});

 @override
 State<PantallaPrincipalMetas> createState() =>
     _PantallaPrincipalMetasState();
}

class _PantallaPrincipalMetasState extends State<PantallaPrincipalMetas> {
 int _metasCompletadas = 0;
 int _metaObjetivo = 5;
 String _nivelProductividad = 'Inicial';

 void _actualizarNivel() {
```

```
  if (_metasCompletadas == 0) {
    _nivelProductividad = 'Inicial';
  } else if (_metasCompletadas < _metaObjetivo) {
    _nivelProductividad = 'En Progreso';
  } else if (_metasCompletadas == _metaObjetivo) {
    _nivelProductividad = '¡Objetivo Alcanzado!';
  } else {
    _nivelProductividad = '¡Sobresaliente (+${_metasCompletadas - _metaObjetivo})!';
  }
}

void _incrementarMeta() {
  setState(() {
    _metasCompletadas++;
    _actualizarNivel();
  });
}

void _decrementarMeta() {
  setState(() {
    if (_metasCompletadas > 0) {
      _metasCompletadas--;
      _actualizarNivel();
    }
  });
}

void _reiniciarContadores() {
  showDialog(
    context: context,
    builder: (BuildContext ctx) {
      return AlertDialog(
        title: const Text('Confirmar Reinicio'),
        content: const Text(
          '¿Desea restablecer todas las metas completadas a cero?',
        ),
        actions: [
          TextButton(
            onPressed: () => Navigator.of(ctx).pop(),
            child: const Text('Cancelar'),
          ),
          FilledButton(
            onPressed: () {
              setState(() {
                _metasCompletadas = 0;
                _actualizarNivel();
```

```
                });
                Navigator.of(ctx).pop();
                ScaffoldMessenger.of(context).showSnackBar(
                  const SnackBar(
                    content: Text('Metas restablecidas exitosamente.'),
                    duration: Duration(seconds: 2),
                  ),
                );
              },
              child: const Text('Reiniciar'),
            ),
          ],
        );
      },
    );
  }

  @override
  Widget build(BuildContext context) {
    final tema = Theme.of(context);
    final colores = tema.colorScheme;
    final porcentaje = (_metasCompletadas / _metaObjetivo).clamp(0.0, 1.0);

    return Scaffold(
      appBar: AppBar(
        title: const Text('FocusTrack Móvil'),
        backgroundColor: colores.primaryContainer,
        foregroundColor: colores.onPrimaryContainer,
        actions: [
          IconButton(
            icon: const Icon(Icons.restart_alt),
            tooltip: 'Reiniciar sesión de metas',
            onPressed: _reiniciarContadores,
          ),
        ],
      ),
      // SafeArea garantiza visualizacion correcta fuera del notch
      body: SafeArea(
        child: SingleChildScrollView(
          padding: const EdgeInsets.symmetric(
            horizontal: 20.0,
            vertical: 24.0,
          ),
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.stretch,
            children: [
```

```
            // Tarjeta de Resumen Ejecutivo
            Card(
              elevation: 3.0,
              shape: RoundedRectangleBorder(
                borderRadius: BorderRadius.circular(16.0),
              ),
              child: Padding(
                padding: const EdgeInsets.all(20.0),
                child: Column(
                  children: [
                    Icon(
                      Icons.track_changes,
                      size: 52.0,
                      color: colores.primary,
                    ),
                    const SizedBox(height: 12.0),
                    Text(
                      'Progreso del Día',
                      style: tema.textTheme.titleMedium?.copyWith(
                        fontWeight: FontWeight.bold,
                      ),
                    ),
                    const SizedBox(height: 8.0),
                    Text(
                      '$_metasCompletadas / $_metaObjetivo',
                      style: tema.textTheme.displayMedium?.copyWith(
                        fontWeight: FontWeight.bold,
                        color: colores.primary,
                      ),
                    ),
                    const SizedBox(height: 12.0),
                    ClipRRect(
                      borderRadius: BorderRadius.circular(8.0),
                      child: LinearProgressIndicator(
                        value: porcentaje,
                        minHeight: 10.0,
                        backgroundColor: colores.surfaceContainerHighest,
                        valueColor: AlwaysStoppedAnimation<Color>(
                          _metasCompletadas >= _metaObjetivo
                              ? Colors.teal
                              : colores.primary,
                        ),
                      ),
                    ),
                    const SizedBox(height: 12.0),
                    Chip(
```

```
                        avatar: Icon(
                          _metasCompletadas >= _metaObjetivo
                              ? Icons.emoji_events
                              : Icons.bolt,
                          color: colores.onSecondaryContainer,
                        ),
                        label: Text(
                          _nivelProductividad,
                          style: TextStyle(
                            fontWeight: FontWeight.bold,
                            color: colores.onSecondaryContainer,
                          ),
                        ),
                        backgroundColor: colores.secondaryContainer,
                      ),
                    ],
                  ),
                ),
              ),
              const SizedBox(height: 24.0),

              // Panel de Informacion Tecnica
              Card(
                elevation: 1.0,
                shape: RoundedRectangleBorder(
                  borderRadius: BorderRadius.circular(12.0),
                  side: BorderSide(
                    color: colores.outlineVariant,
                  ),
                ),
                child: const ListTile(
                  leading: Icon(Icons.info_outline),
                  title: Text('Sesión Móvil Activa'),
                  subtitle: Text(
                    'Renderizado nativo impulsado por el motor Impeller '
                    'y gestionado con Hot Reload.',
                  ),
                ),
              ),
            ],
          ),
        ),
      ),
      // Botones Flotantes de Accion
      floatingActionButton: Row(
        mainAxisAlignment: MainAxisAlignment.end,
```

```
        children: [
          FloatingActionButton(
            heroTag: 'btn_decrementar',
            onPressed: _decrementarMeta,
            tooltip: 'Restar meta completada',
            backgroundColor: colores.surfaceContainerHighest,
            foregroundColor: colores.onSurfaceVariant,
            child: const Icon(Icons.remove),
          ),
          const SizedBox(width: 14.0),
          FloatingActionButton.extended(
            heroTag: 'btn_incrementar',
            onPressed: _incrementarMeta,
            tooltip: 'Completar meta',
            backgroundColor: colores.primary,
            foregroundColor: colores.onPrimary,
            icon: const Icon(Icons.check),
            label: const Text('Completar Meta'),
          ),
        ],
      ),
    );
  }
}
```

Paso 1.5: Selección de Dispositivo y Ejecución de la Aplicación

Existen dos vías principales para ejecutar y depurar la aplicación en su estación de trabajo:

Opción A: Dispositivo Físico Android por Cable USB (Recomendado por Rendimiento)

En su smartphone Android, active las **Opciones de desarrollador** pulsando 7 veces consecutivas sobre **Ajustes > Acerca del teléfono > Número de compilación**.

Active el interruptor **Depuración por USB** (*USB Debugging*).

Conecte su teléfono a la computadora mediante cable de datos USB.

Desbloquee la pantalla de su teléfono y, al aparecer el diálogo de seguridad con la clave criptográfica RSA, marque la casilla **Permitir siempre desde esta computadora** y presione **Aceptar**.

En la terminal de VS Code, compruebe que el teléfono esté vinculado: `powershell adb devices` *Salida esperada:* Aparecerá el serial del dispositivo acompañado de la palabra `device`.

Opción B: Emulador de Android (AVD)

En la esquina inferior derecha de VS Code, haga clic en el selector de dispositivos o presione `Ctrl + Shift + P` y ejecute: **`Flutter: Select Device`**.

Seleccione su emulador previamente creado (ej. `Pixel 8 API 34`).

VS Code levantará el proceso de emulación en una ventana independiente.

Despliegue de la Aplicación

Presione la tecla **`F5`** (o ejecute `flutter run` en la terminal). Durante el primer lanzamiento, Gradle descargará dependencias y compilará los módulos nativos en C++ y Java. Al completarse, verá la aplicación desplegada en la pantalla de su dispositivo móvil.

Paso 1.6: Experimentación del Flujo de Trabajo en Vivo (Stateful Hot Reload)

El desarrollo móvil con Flutter destaca por su aceleración de ciclo de retroalimentación:

Interactúe con la aplicación en su teléfono o emulador pulsando el botón **Completar Meta** 3 veces (el contador marcará `3 / 5`).

En `lib/main.dart`, localice la línea del color semilla: `dart seedColor: const Color(0xFF1A237E), // Azul Índigo` Cámbielo por verde esmeralda: `dart seedColor: const Color(0xFF00796B), // Verde Esmeralda`

Guarde los cambios presionando **`Ctrl + S`**.

Resultado Inmediato: En menos de **800 milisegundos**, la barra superior, los botones y la barra de progreso cambiarán a la tonalidad verde esmeralda **sin perder el estado en memoria** (el contador continúa intacto en `3`).

Parte 2: Depuración y Resolución Exhaustiva de Problemas de Configuración

En esta sección se abordan en detalle los fallos de configuración y virtualización más recurrentes a los que se enfrenta el estudiantado, con procedimientos metódicos para su corrección definitiva.

■ Problema Crítico 1: El Emulador Android No Inicia por Problemas de Virtualización (Hyper-V / WHPX / BIOS SVM / Intel VT-x)

Síntomas y Errores en Consola

Al intentar iniciar el emulador desde Android Studio o VS Code, el proceso se cierra abruptamente tras unos segundos, emitiendo mensajes como: * `The emulator process for AVD Pixel_8_API_34 was killed.` * `Android Emulator requires computer hardware virtualization (VT-x or AMD-V).` * `HAXM is not installed / Hyper-V is disabled.` * `WHX (Windows Hypervisor Platform) is not installed.`

Causa Raíz de Arquitectura

El emulador de Android (QEMU) requiere aceleración por hardware provista por el procesador del anfitrión. Si la tecnología de virtualización de la CPU no está habilitada en la placa madre (BIOS/UEFI), o si en Windows 10/11 coexisten conflictos entre el hipervisor de Windows (**Hyper-V / WSL2 / Docker Desktop**) y los módulos de virtualización heredados (Intel HAXM o AMD Hypervisor Driver), el proceso del emulador colapsa de forma silenciosa.

Procedimiento de Solución Metódica

Siga estos pasos de diagnóstico y activación de forma secuencial:

- [ ] **Paso 1: Verificar el Estado de Virtualización de la CPU en el Administrador de Tareas**
- Presione `Ctrl + Shift + Esc` para abrir el **Administrador de tareas**.

- Ingrese a la pestaña **Rendimiento** y seleccione **CPU**.

  Verifique el campo **Virtualización**:

  - Si indica **Habilitado**, avance al Paso 3.
  - Si indica **Deshabilitado**, continúe con el Paso 2.

  [ ] **Paso 2: Habilitar la Virtualización en el Firmware BIOS/UEFI**

- Reinicie la computadora y presione reiteradamente la tecla de acceso a BIOS (`F2`, `F10`, `F12` o `Supr` según el fabricante del equipo: Dell, HP, Lenovo, ASUS).

  En el menú de configuración avanzada de CPU (*Advanced CPU Settings*):

  - En procesadores **Intel**: Active la opción **Intel Virtualization Technology (Intel VT-x)**.
  - En procesadores **AMD**: Active la opción **SVM Mode (Secure Virtual Machine)**.

  Guarde los cambios (`F10: Save & Exit`) y permita que Windows arranque.

  [ ] **Paso 3: Activar la Plataforma del Hipervisor de Windows (WHPX y Hyper-V)** Para que el emulador moderno de Android funcione en conjunto con Docker Desktop o WSL2 sin conflictos, debe habilitar el hipervisor nativo de Windows:

- Presione `Win + R`, escriba `optionalfeatures.exe` y presione `Enter`.
- En la lista de características, marque obligatoriamente:
  - **Hyper-V** (incluyendo herramientas de administración y plataforma).
  - **Plataforma de máquina virtual** (*Virtual Machine Platform*).
  - **Plataforma del hipervisor de Windows** (*Windows Hypervisor Platform*).
- Presione **Aceptar** y **reinicie su computadora**.

(Alternativa rápida mediante PowerShell con privilegios de Administrador): `powershell # Ejecutar en PowerShell como Administrador Enable-WindowsOptionalFeature -Online -FeatureName HypervisorPlatform -All Enable-WindowsOptionalFeature -Online -FeatureName VirtualMachinePlatform -All`

[ ] **Paso 4: Forzar el Hipervisor en el Gestor de Arranque (BCDEDIT)** Si tras habilitar las características el emulador aún no detecta el hipervisor, fuerce su inicio automático con el comando de arranque de Windows: `powershell # Ejecutar en PowerShell como Administrador y reiniciar equipo bcdedit /set hypervisorlaunchtype auto`

[ ] **Paso 5: Configurar la Tarjeta Gráfica en el AVD (Device Manager)**

- En Android Studio, abra el **Device Manager**.
- En su emulador creado, haga clic en el ícono del lápiz (*Edit*).
- Presione **Show Advanced Settings** y desplace hacia abajo hasta **Emulated Performance**.
- En el campo **Graphics**, cambie la opción de `Automatic` a **Hardware - GLES 2.0**. Esto descarga el procesamiento gráfico sobre su GPU física (NVIDIA, AMD o Intel Iris).

Presione **Finish** y vuelva a encender el emulador.

[ ] **Paso 6: Alternativa Inmediata ante Máquinas con Recursos Limitados** Si su equipo cuenta con 8 GB de RAM o menos y la emulación degrada el rendimiento de la máquina:

> **Conecte su teléfono físico Android por cable USB.**
> Esta solución elimina al 100% el consumo de CPU y memoria RAM asignados a la máquina virtual, brindando 60 a 120 FPS fluidos de manera instantánea.

■ Problema 2: Error "cmdline-tools component is missing" en `flutter doctor`

Síntomas

Al correr `flutter doctor`, se muestra una cruz roja con la siguiente advertencia:

```
[✗] Android toolchain - develop for Android devices
   ✗ cmdline-tools component is missing
     Run `path/to/sdkmanager --install "cmdline-tools;latest"`
```

Solución

1. Abra **Android Studio**.

2. Diríjase a **Settings** (o *Preferences* en macOS) > **Languages & Frameworks** > **Android SDK**.
3. Haga clic en la pestaña **SDK Tools**.
4. Marque la casilla: **Android SDK Command-line Tools (latest)**.
5. Presione **Apply**, acepte la descarga de componentes y haga clic en **Finish**.
6. Vuelva a ejecutar `flutter doctor -v` en su terminal; el error desaparecerá.

■ Problema 3: Error de Licencias de Android no Aceptadas

Síntomas

```
[!] Android toolchain - develop for Android devices
    ! Some Android licenses not accepted. To resolve this, run:
      flutter doctor --android-licenses
```

Solución

Ejecute en la terminal de VS Code el asistente interactivo de licencias:

```
flutter doctor --android-licenses
```

Aparecerá el texto legal de cada SDK instalado. Presione la tecla **`y`** seguida de **`Enter`** sucesivamente hasta recibir la confirmación:

```
All SDK package licenses accepted
```

■ Problema 4: Dispositivo Físico Figura con Estado **`unauthorized`**

Síntomas

Al listar los dispositivos vinculados por USB con `adb devices`, el teléfono figura de la siguiente manera:

```
List of devices attached
9A281FFAZ003    unauthorized
```

Solución

1. En la consola ejecute: `powershell adb kill-server adb start-server`
2. Desconecte el cable USB de la computadora.
3. Desbloquee la pantalla de su teléfono móvil.
4. Vuelva a conectar el cable USB.
5. Aparecerá en la pantalla del teléfono el mensaje:
   "¿Permitir depuración por USB desde esta computadora?".
6. Marque la casilla **"Permitir siempre desde esta computadora"** y pulse **Aceptar**.
7. Verifique nuevamente con `adb devices`; ahora se mostrará con el estado `device`.

■ Problema 5: Restricción de Políticas de Ejecución de PowerShell (`PSSecurityException`)

Síntomas

Al intentar ejecutar comandos como `flutter` o scripts de entorno en la terminal integrada de VS Code, Windows bloquea la ejecución:

```
flutter.ps1 no se puede cargar porque la ejecución de scripts está deshabilitada en este sistema.
+ CategoryInfo          : SecurityError: (:) [], PSSecurityException
```

Solución

Abra PowerShell en modo Administrador y asigne la directiva de ejecución segura para scripts locales:

```
Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned -Force
```

Cierre y vuelva a abrir Visual Studio Code para consolidar el permiso.

■ Problema 6: Incompatibilidad de Versión de Java con Gradle Wrapper

Síntomas

Al compilar con `F5` o `flutter run`, Gradle arroja una excepción:

```
Unsupported class file major version 65 (o 66)
Could not determine the suitability of the Java Virtual Machine.
```

Solución

El wrapper de Gradle utilizado por la plantilla Android de Flutter no soporta Java 22 o versiones de vanguardia no estabilizadas.
1. Verifique que su variable de entorno `JAVA_HOME` apunte estrictamente a **Java 17 LTS** (o Java 21 LTS). 2. En PowerShell verifique: `powershell java -version echo $env:JAVA_HOME` *Ruta de ejemplo válida en Windows:* `C:\Program Files\Eclipse Adoptium\jdk-17.0.x-hotspot`.

Parte 3: Reto Autónomo de Aplicación

Para consolidar las competencias adquiridas en el laboratorio, usted deberá extender la aplicación **FocusTrack Móvil** de manera independiente, satisfaciendo los siguientes requerimientos técnicos:

Especificaciones del Reto

- [ ] **1. Selector de Estado de Prioridad:** Agregue debajo de la tarjeta principal un componente de selección segmentada (`SegmentedButton` o un conjunto de 3 `ActionChip`) que permita al usuario clasificar su sesión en tres niveles:
- `Baja` (Color gris / azul tenue).
- `Media` (Color naranja / ámbar).

  `Alta` (Color rojo / borgoña). Al seleccionar una prioridad, el tema o el subtítulo informativo del panel debe reflejar visualmente la elección seleccionada.

[ ] **2. Límite de Productividad y Bloqueo Seguro:** Implemente una validación que impida incrementar el contador más allá de un valor máximo permitido de **10 metas diarias**. Si el usuario intenta incrementar cuando el contador ya marca `10`, la aplicación no debe aumentar el valor y debe presentar un `SnackBar` con advertencia:
"Ha alcanzado el límite máximo diario de metas (10/10). ¡Excelente jornada!".

[ ] **3. Control de Versiones con Git:** Registre sus avances en el repositorio local de Git mediante commits descriptivos y semánticos siguiendo la convención:

- `git commit -m "feat(mobile): create initial Material 3 dashboard layout"`
- `git commit -m "fix(setup): resolve WHPX virtualization and AVD display"`
- `git commit -m "feat(challenge): add priority segment and maximum goal guard"`

Criterios de Evaluación y Lista de Cotejo

Utilice la siguiente lista para autoevaluar su entrega de laboratorio:

Criterio	Estad	Ponderación
Diagnóstico de Toolchain: Ejecución limpia de `flutter doctor -v` con SDK de Android y VS Code verificados.	`-[]`	20%
Resolución de Entorno: Emulador AVD funcional o dispositivo físico vinculado correctamente mediante ADB.	`-[]`	20%
Arquitectura de Interfaz: Pantalla estructurada con `Scaffold`, `SafeArea`, `Material 3` y sin desbordamientos (*overflow*).	`-[]`	25%

Reactividad Local: Incremento, decremento, diálogo de confirmación y reinicio con `setState()`.	- [ ]	20%
Reto Autónomo: Selector de prioridad y límite máximo de metas funcional.	- [ ]	15%
Total Obtenido:		**100%**

> [!TIP] **Recomendación Profesional:** Acostúmbrese a inspeccionar la pestaña **Debug Console** de VS Code mientras depura su aplicación en vivo. En ella se imprimen los registros de reconstrucción de widgets y los tiempos exactos de Hot Reload.